

Comparativo de Configurações e Solução de Navegação: UD-H1

Este documento apresenta um comparativo técnico e conceitual entre a proposta anterior de controle em malha fechada de alta frequência (encontrada no histórico de planejamento) e a **arquitetura de produção atual** (revertida para a base física estável do backup `Navigation-main` sob demanda da equipe e otimizada via parametrização do Nav2).

1. Visão Geral da Realidade Atual do Projeto

Para evitar riscos elétricos/mecânicos e manter a compatibilidade total com o ecossistema conhecido pela equipe, a base física foi totalmente restaurada. A tabela abaixo detalha onde residem as soluções para os travamentos observados em pista:

Módulo / Funcionalidade	Estado Proposto (Histórico)	Estado Atual (Raiz Ativa / Produção)	Como o Erro de Navegação foi Resolvido
Frequência de Controle	50 Hz (intervalo de 20ms)	10 Hz (intervalo de 100ms)	Mantido o loop original estável e homologado no Arduino pela equipe.
Firmware do Arduino	PI + Feedforward com Anti-Windup	Código Original de Backup	Mantida a leitura baseada em PCINT e TRIM estático do hoverboard.
Driver ROS 2 Python	<code>base_driver_pulse.py</code> com rampas	<code>base_driver.py</code> (Original)	Retorno à cinemática e ao tratamento de eixos estável do backup.

Segurança Ativa	Desativado	Nó <code>safe_stop</code> Integrado	O nó de parada de emergência foi integrado ao launch de navegação para parar o robô a menos de 30 cm de obstáculos frontais.
Navegação (Nav2)	Parâmetros com Bug de Velocidade Lateral	Parâmetros Sintonizados e Corrigidos	Corrigida a velocidade mínima de Y para <code>0.001</code> m/s, impedindo o travamento e aborto de metas no robô diferencial.

2. Correções de Navegação Detalhadas (Ações de Software)

Abaixo estão os trechos e parâmetros internos nos arquivos de configuração do Nav2 (`nav2_params.yaml`, `nav2_params_realsense.yaml` e `nav2_params_sim.yaml`) que resolveram os problemas de navegação autônoma sem alterar uma única linha de código nos drivers da equipe:

A. Limite de Velocidade Lateral (`min_y_velocity_threshold`)

No arquivo original, a velocidade mínima lateral exigida estava configurada como `0.5` m/s. Como o robô diferencial tem velocidade lateral nula ($v_y = 0$), a condição nunca era satisfeita, travando o planejador.

```
# Caminho: src/udh1_mapping/config/nav2_params.yaml (e realsense/sim)
controller_server:
  ros__parameters:
    min_x_velocity_threshold: 0.001
    min_y_velocity_threshold: 0.001 # Corrigido de 0.5 para 0.001
    min_theta_velocity_threshold: 0.001
```

B. Tolerância das Transformadas (TF Latency)

Aumentamos o tempo limite das transformadas para tolerar pequenos atrasos na recepção das mensagens vindas da comunicação serial com o microcontrolador:

```
# AMCL (Localizador)
amcl:
  ros__parameters:
    transform_tolerance: 1.5    # Aumentado de 1.0 para 1.5

# DWB (Planejador de Trajetória Local)
FollowPath:
  transform_tolerance: 1.0    # Aumentado de 0.2 para 1.0
```

C. Parâmetros do Verificador de Progresso (SimpleProgressChecker)

Evita que manobras lentas ou de encaixe em passagens estreitas gerem cancelamentos inesperados de metas:

```
progress_checker:
  plugin: nav2_controller::SimpleProgressChecker
  required_movement_radius: 0.3    # Reduzido de 0.5 para 0.3
  movement_time_allowance: 15.0    # Estendido de 10.0 para 15.0 segundos
```

D. Remoção de Comportamento de Rotação Perigoso (**spin recovery**)

Desativamos a tentativa automática de girar 360° em locais confinados para evitar colisões físicas com o chassi do robô:

```
recoveries_server:
  ros__parameters:
    recovery_plugins: [backup, wait]    # Removido o plugin 'spin'
```

3. Conclusão

Com a estratégia atualizada, a equipe do **UNIP Droidians** possui um código robusto de navegação autônoma cujo controle de baixo nível e hardware é **100% idêntico** ao que eles já sabiam operar e manter. O ganho e a resolução dos erros de travamento foram obtidos estritamente através do ajuste e sintonia de engenharia dos parâmetros do Nav2 e do AMCL.